Project - Low Level Design

on

Dockerized Entertainment React Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AMAN SHARMA	EN22CS301109
2	ANANYA BHATIA	EN22CS301117
3	ANKIT NAGAR	EN22CS301134
4	ANUSHA SINGH PANWAR	EN22CS301179
5	ARPIT PATIDAR	EN22CS301199

Group Name: Group 02D2

Project Number: DO-14

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year: Jan-June 2026

Table of Contents

1.Introduction

The Dockerized Entertainment React Service is a web-based application designed to provide users with an interactive platform to explore entertainment content such as movies and media recommendations. The application is built using React for creating a dynamic and responsive user interface, and it is containerized using Docker to ensure consistency across different environments.

The primary goal of this system is to deliver a seamless user experience by enabling users to browse, search, and view entertainment-related content efficiently. The use of containerization simplifies deployment, improves scalability, and ensures that the application runs reliably regardless of the host system.

1.1 Scope of the Document

This document focuses on the detailed internal design of the **Dockerized Entertainment React Service**. It covers the structural and functional aspects of the system, including component-level architecture, module interactions, API communication, and containerized deployment.

The scope of this document includes:

- Detailed description of frontend components developed using React
- Interaction between UI components and backend/external APIs
- Design of reusable modules and services
- API request and response handling
- Containerization setup using Docker for deployment
- Error handling and performance considerations

1.2 Intended Audience

This document is intended for:

1. **Project Evaluators / Faculty:** This document provides detailed technical insights into the system's internal design, helping evaluators assess the depth of implementation, modular structure, and adherence to software engineering practices.

2. **DevOps / Deployment Engineers:**
   Individuals responsible for deployment can refer to this document to understand containerization, environment configuration, and how the application is packaged and executed using Docker.

3. **Future Maintainers:**
   Any developer who works on enhancing or maintaining the system in the future can use this document as a reference to understand existing modules, data flow, and integration points.

4. **Developers:** Frontend developers working with **React** can use this document to understand component structure, state management, API integration, and UI behavior.

5. **Technical Stakeholders:**
   Anyone interested in the technical architecture and internal working of the application can use this document to gain a clear understanding of how the system operates at a detailed level.

1.3 System Overview

The **Dockerized Entertainment React Service** is a web-based application that allows users to browse and explore entertainment content such as movies through an interactive and user-friendly interface.

The system is going to be developed using **React**, which enables the creation of reusable UI components and ensures a dynamic user experience. The application interacts with backend or external APIs to fetch and display real-time content data based on user actions such as searching or selecting categories.

To ensure consistent deployment and ease of scalability, the application is containerized using **Docker**. This allows the system to run uniformly across different environments without dependency issues.

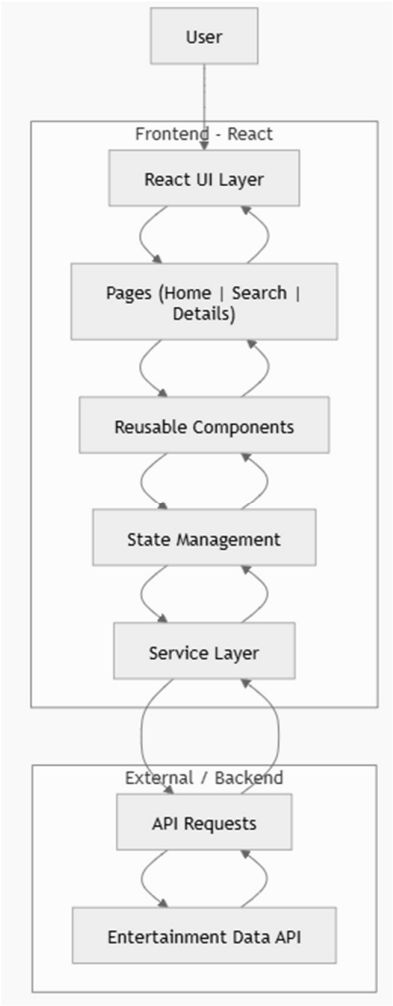
Overall, the system follows a modular architecture where the frontend handles user interaction, API services manage data communication, and Docker ensures smooth deployment and execution of the application.

2. System Design

2.1 Application Design

The application design is based on a component-driven architecture implemented using **React**. The system is divided into multiple layers to ensure clarity and maintainability.

1. **Presentation Layer (UI):**
   Handles user interaction and displays content. It includes components such as Home, Search, Movie List, and Detail View.
2. **Service Layer:**
   Responsible for handling API calls. It manages communication between the frontend and external/backend services, processes responses, and sends structured data to UI components.
3. **State Management:**
   Maintains application data such as user input, search results, and selected content. This ensures smooth data flow between components.
4. **Routing Mechanism:**
   Enables navigation between different pages or views within the application without reloading the page.



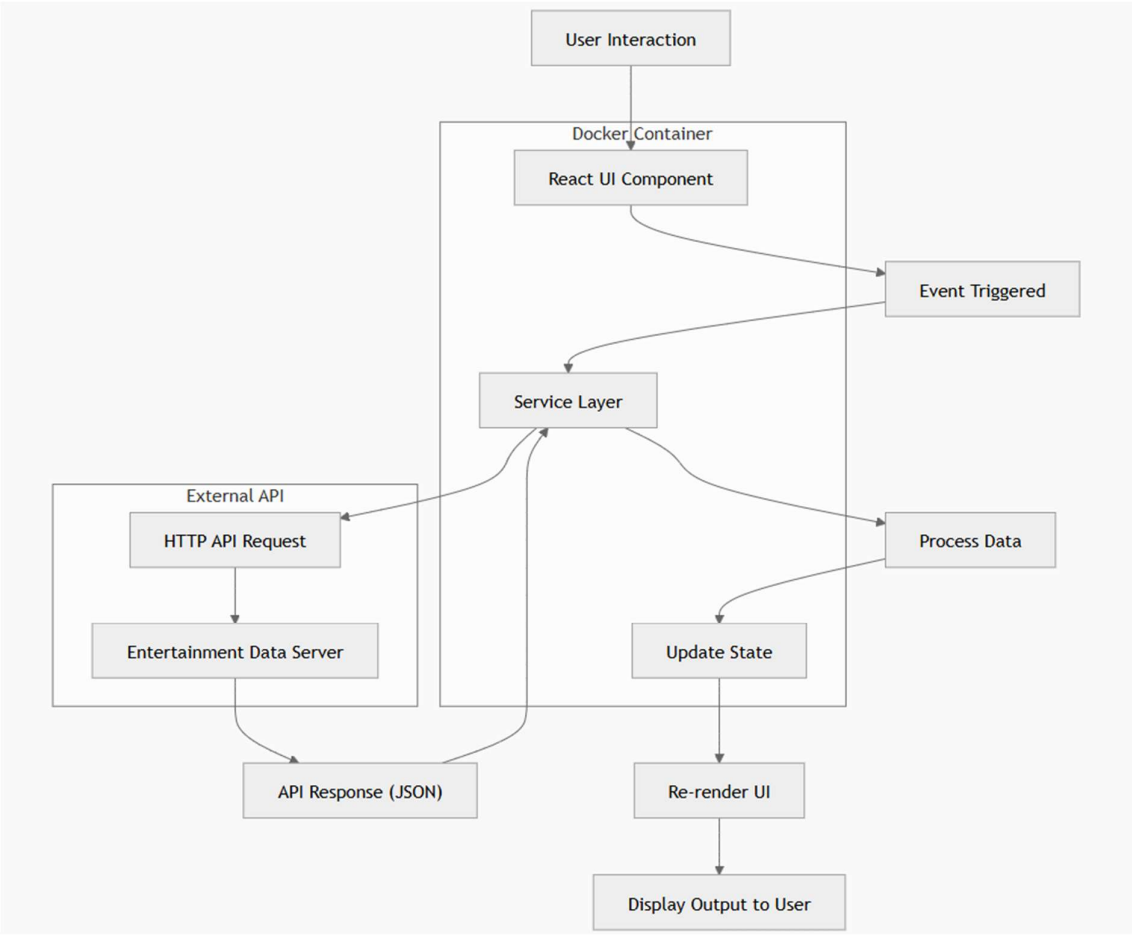
2.2 Process Flow

The system processes user requests through an intelligent loop.

Steps:

1. The process begins when a user interacts with the application through the UI built using **React**, such as entering a search query or selecting a movie.
2. The user action is captured by the respective React component (e.g., Search or Home component), which triggers an event.
3. The component sends the request to the service layer, where API handling logic is implemented.
4. The service layer makes an HTTP request to an external or backend entertainment API to fetch relevant data.
5. The API processes the request and returns the response in JSON format.

6. The service layer receives and processes the response, then forwards the data to the state management system.
7. The state is updated, and the updated data is propagated to the UI components.
8. The UI re-renders dynamically and displays the requested content (movies, shows, etc.) to the user.
9. All these operations are executed within a containerized environment using **Docker**, ensuring consistent performance and deployment.



2.3 Information Flow

1. The flow begins when the user provides input (such as a search query or selection) through the UI developed using **React**.

2. The input data is captured by the UI components and passed to the service layer in a structured format.

3. The service layer converts this input into an HTTP request and sends it to the external or backend API.

4. The API processes the request and returns data in a structured format (typically JSON).

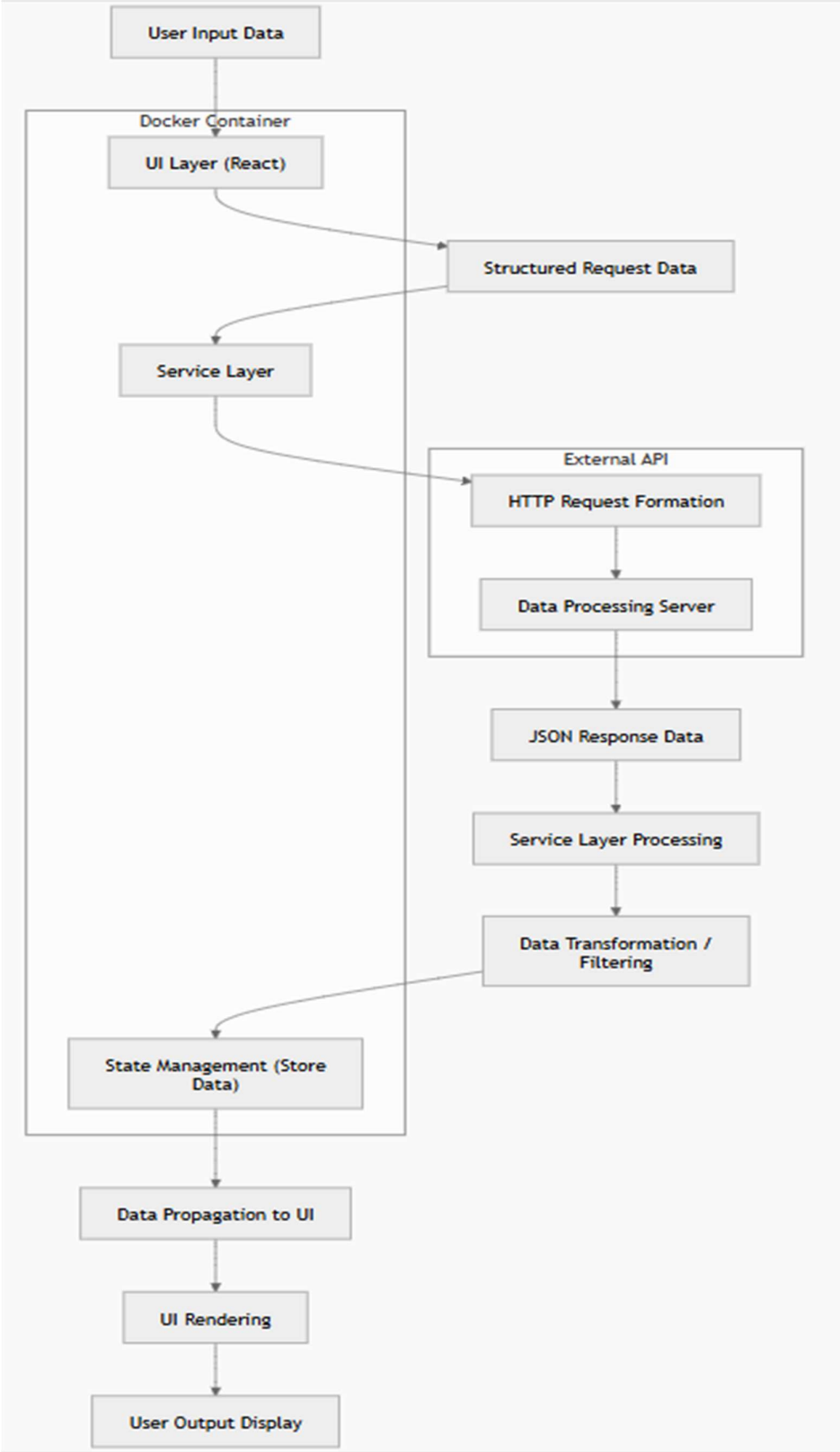
5. The received data flows back to the service layer, where it is filtered, formatted, or transformed as required.

6. The processed data is then passed to the state management system, which stores and manages the application data.

7. The updated state is propagated to the UI components, ensuring that only relevant and updated data is displayed.

8. Finally, the UI renders the information and presents it to the user in a user-friendly format.

9. This entire data flow is managed within a containerized environment using **Docker**, ensuring consistency and efficient data handling.


User Input Data
Docker Container
UI Layer (React)
Structured Request Data
Service Layer
External API
HTTP Request Formation
Data Processing Server
JSON Response Data
Service Layer Processing
Data Transformation / Filtering
State Management (Store Data)
Data Propagation to UI
UI Rendering
User Output Display

2.4 Components Design

2.4.1 App Component

- Acts as the root component of the application
- Manages routing and overall layout
- Connects all other components

2.4.2 Header / Navigation Component

- Provides navigation across different pages
- Contains links such as Home, Search, and Categories

2.4.3 Home Component

- Displays trending or featured entertainment content
- Fetches initial data through the service layer

2.4.4 Search Component

- Accepts user input for searching content
- Sends search queries to the service layer

2.4.5 Content List Component

- Displays a list of movies or shows
- Receives data from state management
- Reusable for different categories

2.4.6 Content Detail Component

- Displays detailed information about selected content
- Uses data passed from the Content List or state

2.4.7 Service Layer Component

- Handles API calls and data fetching
- Communicates with external/backend APIs
- Processes and returns structured data

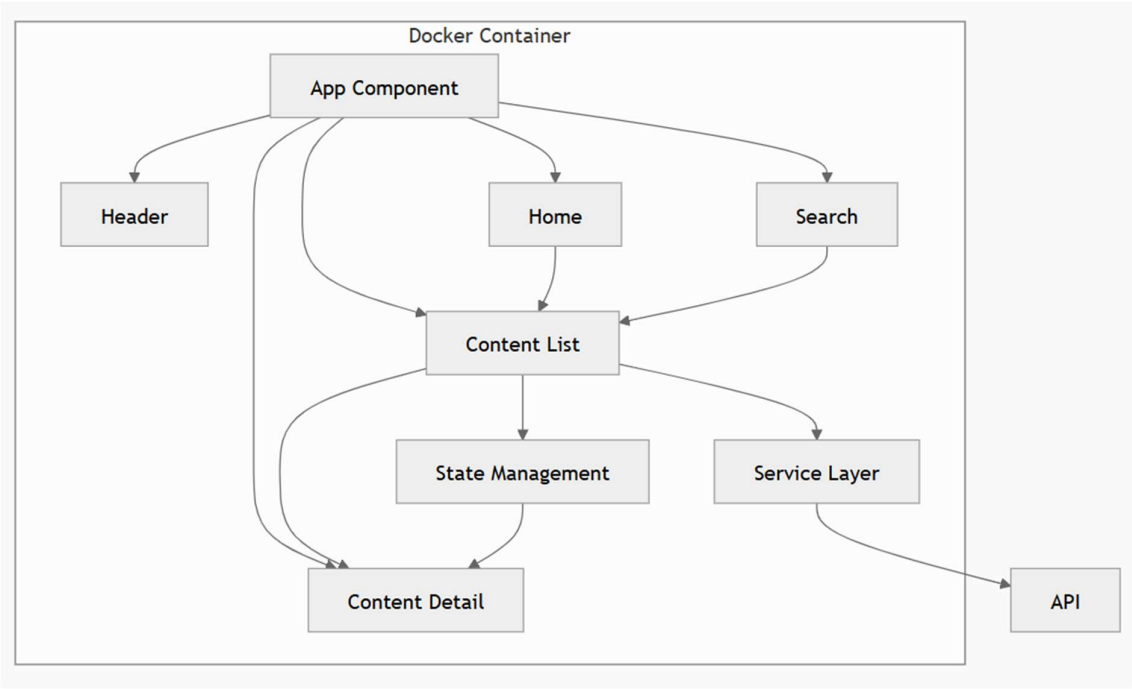
2.4.8 State Management

- Stores application data (search results, selected content)

- Ensures data flow between components
- Triggers UI updates when data changes

2.4.9 Docker Container

- Packages the entire application using **Docker**
- Ensures consistent deployment across environments



2.5 Key Design Considerations

2.5.1 Modularity

The application is divided into independent and reusable components using **React**. This improves code maintainability and allows easy updates or enhancements.

2.5.2 Scalability

The system is designed to handle increasing user requests by efficiently managing API calls and component rendering. The use of **Docker** enables easy scaling across different environments.

2.5.3 Performance Optimization

Efficient data fetching, minimized re-rendering of components, and proper state management are considered to ensure fast response times and smooth user experience.

2.5.4 Reusability

Reusable components such as content cards, lists, and navigation elements are designed to avoid code duplication and improve development efficiency.

2.5.5 Maintainability

The code structure is organized and follows best practices, making it easier for developers to understand, debug, and modify the system in the future.

2.5.6 Data Handling

Proper handling of API responses (JSON data), including validation and transformation, ensures accurate and reliable data display.

2.5.7 Error Handling

The system includes mechanisms to handle API failures, invalid inputs, and network issues to prevent application crashes and improve reliability.

2.5.8 Deployment Consistency

The application is containerized using **Docker**, ensuring consistent behavior across development, testing, and production environments.

2.6 API Catalogue

2.6.1 Fetch Trending Content API

- **Endpoint:** /api/trending
- **Method:** GET
- **Description:** Retrieves a list of trending movies or shows
- **Request Parameters:** None
- **Response:** List of trending content (JSON format)

2.6.2 Search Content API

- **Endpoint:** /api/search
- **Method:** GET
- **Description:** Searches for movies or shows based on user input
- **Request Parameters:**
  - query (string): Search keyword
- **Response:** List of matching results (JSON format)

2.6.3 Get Content Details API

- **Endpoint:** /api/details/{id}
- **Method:** GET
- **Description:** Retrieves detailed information about a selected movie or show
- **Request Parameters:**
  - id (string): Unique identifier of the content
- **Response:** Detailed content information (JSON format)

2.6.4 Fetch Category-wise Content API

- **Endpoint:** /api/category/{type}
- **Method:** GET
- **Description:** Retrieves content based on categories such as Action, Comedy, etc.
- **Request Parameters:**
  - type (string): Category name
- **Response:** List of content based on category (JSON format)

2.6.5 Error Handling API Response

- **Description:** Handles API errors such as invalid requests or server issues
- **Response Format:**

{
"error": "Unable to fetch data",
"status": 500
}

3. References

References (IEEE Format)

[1] React, "React – A JavaScript library for building user interfaces," Meta, 2024. [Online]. Available: https://react.dev/

[2] Docker, "Docker Documentation," Docker Inc., 2024. [Online]. Available: https://docs.docker.com/

[3] Node.js, "Node.js Documentation," OpenJS Foundation, 2024. [Online]. Available: https://nodejs.org/

[4] REST API, "RESTful Web Services," 2024. [Online]. Available: https://restfulapi.net/

[5] Axios, "Axios HTTP Client Documentation," 2024. [Online]. Available: https://axios-http.com/

[6] The Movie Database (TMDb) API, "TMDb API Documentation," 2024. [Online]. Available: https://developers.themoviedb.org/

[7] JavaScript, "JavaScript Guide," Mozilla Developer Network, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[8] HTML, "HTML Standard," WHATWG, 2024. [Online]. Available: https://html.spec.whatwg.org/

[9] CSS, "CSS Documentation," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS

[10] Git, "Git Documentation," 2024. [Online]. Available: https://git-scm.com/docs

[11] GitHub, "GitHub Docs," 2024. [Online]. Available: https://docs.github.com/

[12] Nginx, "NGINX Documentation," F5 Inc., 2024. [Online]. Available: https://nginx.org/en/docs/